NFL In-Game Win Probability vs Kalshi: Project To-Do

Stochastic Modeling final project. Proposal due Thursday, October 29, 11:59 PM (PDF on Canvas, one per team, list all member names).

Research question

Are Kalshi's live NFL win probabilities calibrated against an absorbing Markov chain built on bucketed game states, and can discrepancies between the two be exploited after fees without a high risk of ruin?

Before Thursday (proposal)

- [ ] Confirm team size and split contributions (data pipeline, Markov model, calibration analysis, betting simulation, writing)
- [ ] Run the data access checks in the next section and record the numbers (games found, seasons, price resolution)
- [ ] Write the 1 to 2 page proposal: question, mathematical model, data sources, methods, contributions, references
- [ ] Mention both required parts: detailed stochastic model (absorbing chain, Ch. 5 and 6) and analysis on real data with code
- [ ] Cite prior work (see Prior work below)
- [ ] Add the AI tool disclosure line (syllabus requires an appendix on the final report; mention intent in the proposal)
- [ ] Export to PDF, include all names, submit one copy on Canvas

Data access checks (VS Code and Codex)

- [ ] Kalshi: call GET /historical/cutoff and note the cutoff timestamps
- [ ] Kalshi: list settled KXNFLGAME markets (live and historical endpoints), count games and seasons
- [ ] Kalshi: pull trades or candlesticks for one game and check how dense the in-game price path is

- [ ] Kalshi: record bid, ask, and spread behavior during play; decide on midpoint vs last trade
- [ ] nflverse: load play-by-play for one 2025 game (nfl_data_py or nflreadpy; confirm the package name on PyPI)
- [ ] nflverse: inspect time_of_day, game_date, total_home_score, total_away_score, yardline_100, down, ydstogo, game_seconds_remaining, wp, vegas_wp
- [ ] Resolve the time_of_day timezone question (docs disagree on UTC vs local) by comparing with the known kickoff time
- [ ] Check how many of the Kalshi games have play-by-play with usable timestamps
- [ ] Start a live price logger now as a backup (Kalshi public endpoints are keyless); log every 30 to 60 seconds during games, store raw JSON plus the local receive time

Data pipeline

- [ ] Match Kalshi events to nflverse games (ticker date is Eastern game date; read teams from the market subtitle, not by splitting the ticker)
- [ ] Build a per-play table: wall-clock time, score, field position, down and distance, time left, possession, Kalshi midpoint nearest each play
- [ ] Estimate the clock offset per game by aligning price jumps with scoring plays
- [ ] Use time windows (price before the snap vs 1 to 2 minutes after) instead of exact matching
- [ ] Handle edge cases: missing times, deleted plays, halftime gaps, overtime, games that cross midnight UTC
- [ ] Save cleaned data to disk so the analysis does not depend on live API calls

Model

- [ ] Define state buckets: score differential, time remaining, field position, down and distance, possession
- [ ] Add the initial strength bucket from the last pregame Kalshi price before kickoff (for example 5 buckets)
- [ ] Start with fewer variables (score, time, possession, field position) and add down and distance only if the data supports it
- [ ] Consider a drive-level chain as a fallback if the play-level state space is too sparse
- [ ] Estimate the transition matrix from play-by-play with smoothing for sparse states

- [ ] Make win and loss the absorbing states; compute win probabilities with the fundamental matrix (Ch. 6)
- [ ] Sanity check against nflfastR wp and vegas_wp before comparing to Kalshi
- [ ] Split data by time: fit on earlier games, test on later games

Evaluation

- [ ] Calibration plot: Kalshi midpoint vs observed win rate in probability bins
- [ ] Same plot for the Markov chain and for nflfastR wp
- [ ] Compare changes: does the market move after each play by the amount the model predicts?
- [ ] Metrics: Brier score, log loss, calibration slope
- [ ] Break results down by game time (early vs late) and by state bucket
- [ ] Report uncertainty (bootstrap by game, since plays within a game are correlated)

Betting layer

- [ ] Define a signal: bet when model and market differ by more than a threshold after fees and spread
- [ ] Check Kalshi's current fee schedule (one blog cites a 7% coefficient in the taker formula; verify on Kalshi's site)
- [ ] Limit to one bet per game, or model the correlation between bets in the same game
- [ ] Simulate bankroll as a random walk: flat staking vs Kelly vs fractional Kelly
- [ ] Compare theoretical ruin probability (Ch. 7) with simulation
- [ ] Optional stretch: MDP for bet sizing (Ch. 14, value iteration)
- [ ] Paper trading only, no real money

Prior work to read and cite

- [ ] Pelechrinis, iWinRNFL: a logistic regression in-game win probability model (arXiv 1704.00197)
- [ ] Moshrefi, Prices, Probabilities, and Parlays: Systematic Bias in Sports Prediction Markets (arXiv 2607.14430); uses 23 million Kalshi moneyline trades
- [ ] Decomposing Crowd Wisdom: Domain-Specific Calibration Dynamics in Prediction Markets (arXiv 2602.19520)

- [ ] Makers or Takers: The Economics of the Kalshi Prediction Market (GWU working paper)
- [ ] nflfastR documentation and the nflverse win probability models
- [ ] Classic football Markov chain and expected points papers (search for these; I have not verified specific titles)

Timeline

- Oct 29: proposal due
- By Nov 5: data checks done, pipeline working on a few games
- By Nov 12: first Markov chain and calibration plot; first review meeting
- Nov 24: in-class midterm (30 minutes); plan lighter project work that week
- By Dec 3: all three review meetings done; betting simulation finished
- Dec 8: final report and in-class presentation

Risks and fallbacks

- Too few Kalshi games with trades data: use the live logger as the main dataset, or add college football or NBA
- Timestamps too noisy: use drive-level states and wider time windows, or a paid timestamp source if a student plan exists
- State space too sparse: fewer variables, coarser buckets, or smoothing toward a parametric model
- No edge after fees: report it as the finding; the calibration and ruin analysis still stand

Notes

- AI tools are allowed on the project but must be disclosed in a short appendix, and every line of code and every statement must be explainable by the team
- Keep code comments focused on functionality, input, and output